Let's dissect this query. The `bool` declaration inside the `query` object indicates that this is a `bool` query. The `bool` object is then wrapped with a `must` clause with the criterion to match the given search word, `TV`. When we execute this query, a few TVs are returned, as expected (the output is omitted here for brevity).

### A match query is a bool query

The `match` query we've worked with so far is a type of Boolean query. For example, the `bool` query in listing 11.2 can be rewritten as a `match` (full-text) query to fetch TVs. The following leaf query returns the same TVs returned by the previous `bool` query:

```
GET books/_search
{
  "query": {
    "match": {
      "author": "Joshua"
    }
  }
}
```

You may be inclined to use a `match` query for a simple (single) criterion, but alas, there's more complexity in the real world; so, you may have to lean on the `bool` query.

## 11.3.3 Enhancing the must clause

Fetching TVs isn't that exciting, is it? Let's make our query a bit more interesting. In addition to fetching TVs, let's add a condition: fetch only TVs whose value is in a certain price range. We need to join two queries to achieve what we want.

This requirement asks us to use a `must` clause with two `match` queries: one to match on the `product` field and another to match on `price`. Remember, the `must` clause accepts an array of leaf-level queries. While a `match` query is sufficient to search on the type of product, we can add a `range` query to a `bool` query to fetch all TVs in a certain price range.

**Listing 11.3 Finding TVs within a price range**

```
GET products/_search
{
  "query": {
    "bool": {
      "must": [
        {
          "match": {
            "product": "TV"
          }
        },
        {
          "range": {
```

The must query with two individual leaf queries

A match query to find TVs

A range query with a price range